

Themes con Compose y Material 3

Aplicación Woof · Código, pruebas y evidencias

Axel Adrián Gaviña González

01 / Objetivo

Personalizar una aplicación Android con Jetpack Compose y Material 3 y comprobar el resultado en un dispositivo real. La práctica adapta el proyecto Woof de Android Developers; conserva sus datos, fotografías y licencias.

02 / Entorno utilizado

PC con Windows y Android Studio. Gradle 9.5.0, Android Gradle Plugin 9.3.3, Kotlin/Compose Compiler 2.2.10 y Compose BOM 2026.02.01. SDK de compilación 37, targetSdk 35 y minSdk 24. La ejecución se realizó en un Xiaomi Redmi 9 (M2004J19C), Android 11 / API 30, conectado por USB a la PC.

03 / Desarrollo realizado

Parte	Implementación
Color	Paletas verde clara y oscura en Color.kt y Theme.kt. Selección según el tema del sistema.
Forma	Fotografías circulares; tarjetas con esquinas superior derecha e inferior izquierda de 16 dp.
Tipografía	Abril Fatface para el título; Montserrat en nombres y edades. Fuentes incluidas como recursos locales.
Interfaz	Scaffold, barra superior centrada con logo, LazyColumn y nueve perros. Previews claro y oscuro.
Ajustes	Compatibilidad con herramientas instaladas, barras del sistema y edades en español con plurales.

Una interfaz, dos temas

Capturas reales obtenidas desde la PC mediante ADB, con Woof ejecutándose en el Redmi. No son imágenes del tutorial ni simulaciones.

TEMA CLARO

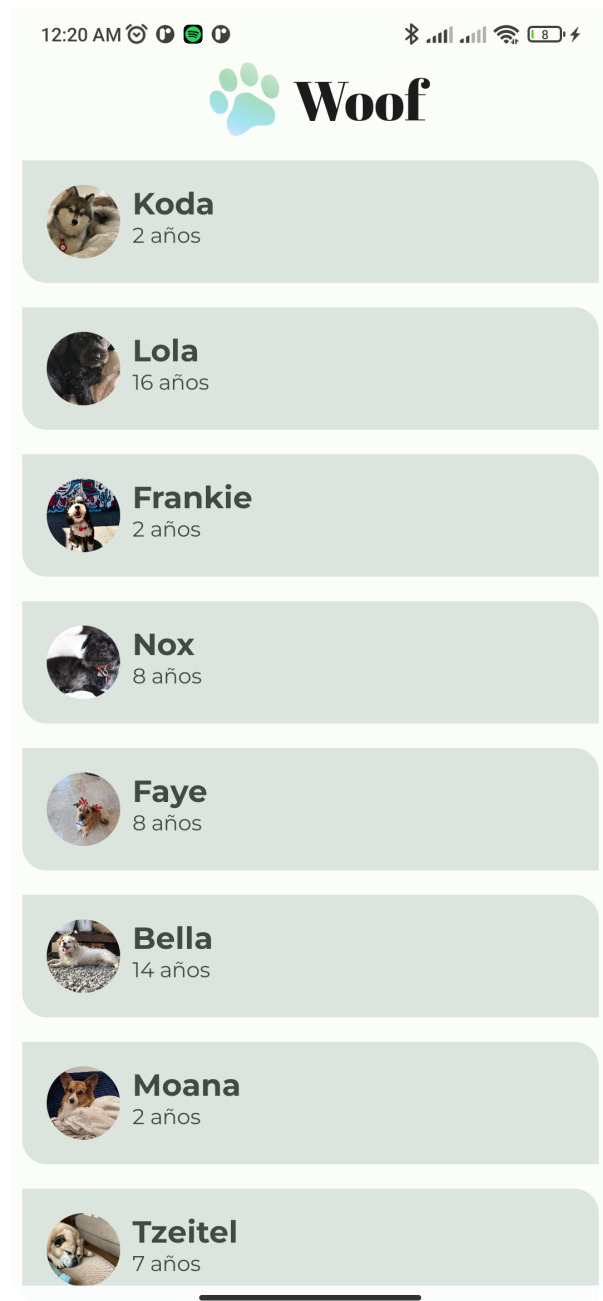


Figura 1. Fondo claro, tarjetas verdes suaves y texto oscuro.

TEMA OSCURO

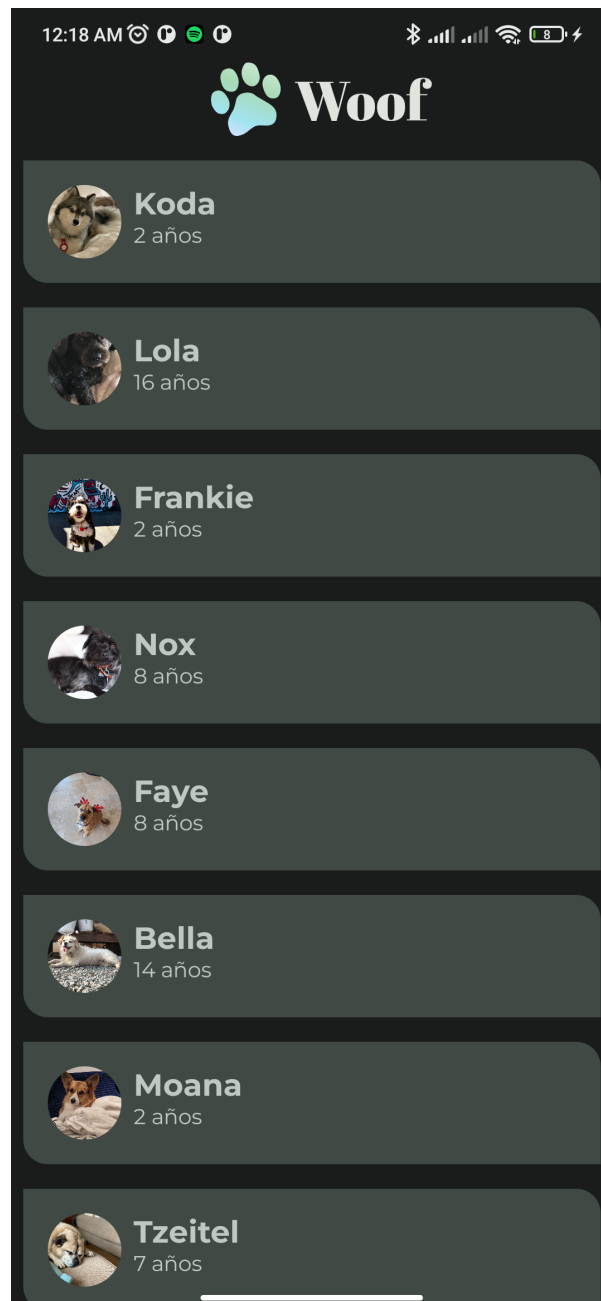


Figura 2. Fondo oscuro, tarjetas y texto ajustados al tema del sistema.

En ambas capturas se distinguen el logo, el título con Abril Fatface, los nombres con Montserrat Bold y el recorte circular de las fotografías.

Lista y resultados de prueba

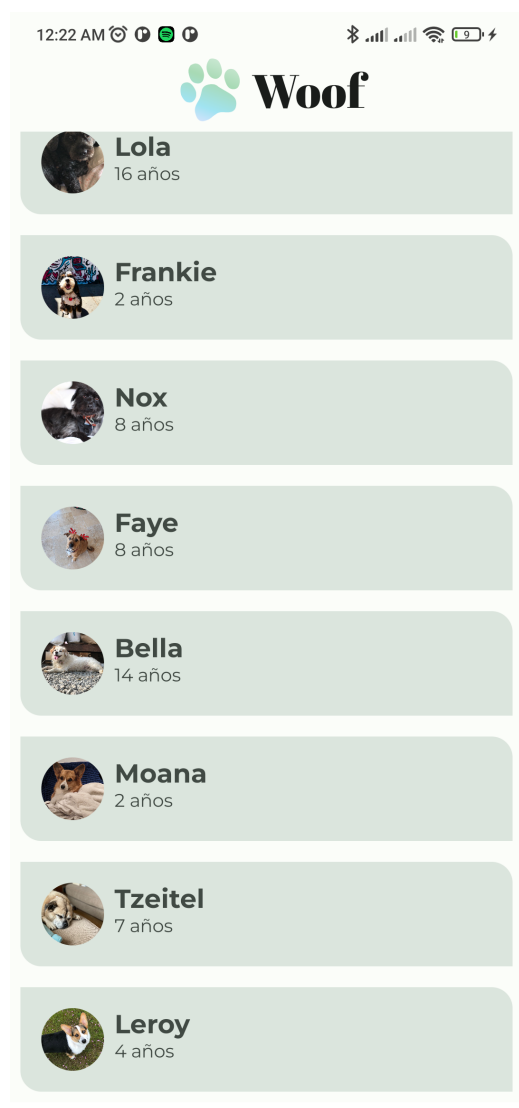


Figura 3. Leroy aparece al final; la barra Woof permanece visible. El desplazamiento lo realizó manualmente el usuario porque el teléfono bloquea la inyección de gestos por ADB.

Compilación: correcta

Se generó el APK de depuración. La validación final ejecutó assembleDebug y lintDebug y terminó con BUILD SUCCESSFUL en 30 s.

Lint: 0 errores

Se conservaron 28 advertencias, principalmente sobre dependencias, recursos e iconos. Se adjunta el reporte completo; no se desactivó la validación.

Instalación e inicio: correctos

ADB devolvió Success en la instalación y Status: ok al abrir com.example.woof/.MainActivity.

Temas y lista: verificados

Se capturaron los temas claro y oscuro y el final de la lista. Se restauró el tema oscuro que tenía originalmente el teléfono.

Color dinámico: no verificado

La ruta de color dinámico está implementada para API 31 o superior y desactivada por defecto. El Redmi usa API 30, por lo que esta prueba requiere otro dispositivo.

Trazabilidad

Las capturas corresponden al commit local 394c3bb. Después se completaron traducciones no visibles y la categoría plural many para resolver Lint (77ded8e), sin cambiar el diseño. Los registros de compilación y ejecución se incluyen en docs/evidencias/.

Conclusiones

Tema centralizado

Definir colores, formas y tipografía en MaterialTheme permite mantener coherencia entre los componentes. El cambio de modo del sistema actualizó la apariencia de Woof sin sustituir la estructura de la pantalla.

Resultado visual

Las capturas muestran una jerarquía clara: título, nombre y edad. Las tarjetas separan los elementos y las fotos circulares conservan su proporción gracias al recorte. La lista permite consultar los nueve registros y mantiene la barra superior visible.

Validación y alcance

Compilar y ejecutar son comprobaciones distintas: se verificó la generación del APK en Windows y su ejecución en el teléfono conectado. No se utilizó un emulador. Quedan fuera de esta evidencia el color dinámico en Android 12+, la rotación y pruebas formales con lector de pantalla.

Cómo reproducir la práctica

1. Clonar el repositorio y abrir su carpeta raíz en Android Studio.
2. Usar JDK 17 o superior compatible con Gradle 9.5, SDK 37 y Build Tools 36.0.0. Sincronizar las dependencias.
3. Ejecutar `gradlew.bat :app:assembleDebug :app:lintDebug` en Windows. La primera ejecución puede requerir Internet.
4. Instalar `app/build/outputs/apk/debug/app-debug.apk` en Android 7 o posterior. Abrir Woof, alternar el tema del sistema y desplazar la lista hasta Leroy.

Código e historial de cambios

github.com/Axel421-yn/woof-compose-material3

El historial separa el proyecto inicial, colores, formas, fuentes, interfaz, adaptación de compilación y correcciones. El repositorio incorpora el informe, capturas y registros.

Fuentes y atribución

Android Developers: Temas de Material con Jetpack Compose

Google Developer Training: código y recursos originales Woof

Se conservaron LICENSE (Apache 2.0) y ASSETS_LICENSE.txt (fuentes SIL Open Font License).